

Unit B2: Programming (HL Extension)

Sub-topics: B2.4.4 & B2.4.5 (Recursion & Advanced Algorithms)

1. Syllabus Scope & Learning Outcomes

These topics are for Higher Level students only and are often tested in Paper 1 Section B or Paper 2.

Syllabus Point	Command Term	Student Expectation (The "Do")
B2.4.4	Explain	Define recursion, Base Case, and General Case. Compare recursion vs. iteration.
B2.4.5	Construct / Trace	Write and trace recursive algorithms (Factorial, Fibonacci, Binary Tree traversals).
B2.4.5	Trace	Trace the execution of Quicksort (partitioning and pivoting).

2. Key Terminology & Definitions

- **Recursion:** A function that calls itself to solve a smaller instance of the problem.
- **Base Case:** The condition that stops the recursion (prevents infinite loops).
- **General Case (Recursive Step):** The part of the function where the logic calls itself with modified parameters.
- **Stack Overflow:** An error caused by too many recursive calls (infinite recursion) filling up the memory stack.
- **Divide and Conquer:** A strategy where a problem is split into sub-problems (used in Quicksort).
- **Pivot:** The element selected in Quicksort to partition the list into "smaller than" and "larger than" sub-lists.

3. Core Content & Exam Actions

B2.4.4: Recursion Fundamentals

Theory:

- **Structure:** Every recursive function *must* have:
 1. **Base Case:** Returns a value without calling the function again (e.g., if $n == 1$: return 1).
 2. **Recursive Call:** Moves the state closer to the base case (e.g., return $n * \text{factorial}(n-1)$).
- **Memory:** Recursion uses the "Call Stack". Each call adds a new "frame" to the stack. This makes recursion memory-intensive compared to iteration (loops).
- **Winding/Unwinding:**
 - *Winding:* Pushing calls onto the stack.
 - *Unwinding:* Popping calls off the stack and resolving values.

Student Exam Action:

- **Comparison:** Explain why recursion is more "elegant" (less code) but less memory-efficient than loops.
- **Debugging:** Identify why a specific recursive function causes a stack overflow (usually a missing or unreachable base case).

B2.4.5: Recursive Algorithms

Theory:

- **Factorial:** $n! = n \times (n - 1)!$
- **Fibonacci:** $\text{fib}(n) = \text{fib}(n - 1) + \text{fib}(n - 2)$. (Note: This is very inefficient recursively due to repeated calculations).
- **Quicksort:**
 - Select a **Pivot**.
 - **Partition** the list: Items $<$ Pivot go left, Items $>$ Pivot go right.
 - Recursively sort the left and right sub-lists.
 - Time Complexity: Average $O(n \log n)$, Worst Case $O(n^2)$.
- **Tree Traversals:**
 - **Pre-order:** Root $\rightarrow$ Left $\rightarrow$ Right.
 - **In-order:** Left $\rightarrow$ Root $\rightarrow$ Right.
 - **Post-order:** Left $\rightarrow$ Right $\rightarrow$ Root.

Student Exam Action:

- **Coding:** Write a Python function `factorial(n)` that uses recursion.
- **Tracing Quicksort:** Given a list [16, 13, 4, 6, 22], show the state of the list after the first partition using the first element as the pivot.
- **Tracing Trees:** Given a diagram of a Binary Tree, list the nodes in the order they are

visited for a Post-order traversal.

4. Common Pitfalls & Examiner Tips

- **Pitfall (Infinite Recursion):** Students often forget to change the parameter in the recursive call (e.g., calling `factorial(n)` instead of `factorial(n-1)`). This causes an infinite loop.
- **Pitfall (Quicksort):** In a trace, students often sort the entire list in one step. You must show the **partitioning** step where the pivot moves to its correct final position, splitting the list.
- **Tip (Tree Traversal):**
 - *Pre* = Process Root *before* children.
 - *In* = Process Root *in-between* children.
 - *Post* = Process Root *after* children.
- **Tip (Efficiency):** If asked to choose between Iteration and Recursion for a mobile app with limited memory, choose Iteration (avoids stack overflow).